

Experiments for AET-RAG

Reproducible code for the experimental section of *Adaptive Event-Triggered Synchronization in Logistics Digital Twins: A RAG-Driven Framework for Unstructured Data*.

Layout

```
experiments/ requirements.txt config.yaml src/ download_data.py # fetch SVRPBench (HF
or GitHub) or fall back to synthetic data_loader.py # Instance dataclass + SVRPBench/synthe
loader event_generator.py # numerical disruption -> driver-style textual events rag_extract
# Oracle + optional OpenAI LLM extractor (pydantic-validated) aet_controller.py # U/S/R,
D_t, theta_t, trigger decision ortools_solver.py # CVRPTW wrapper around Google OR-Tools
baselines.py # static / continuous / AET-RAG scenarios metrics.py # per-scenario + aggregat
metrics run_experiments.py # CLI orchestrator plot_results.py # figures referenced in
paper/root.tex data/ # SVRPBench cache (populated by download_data) outputs/ raw/ # per-ins
event logs, per-scenario per-event logs metrics/ # summary.csv, aggregate.csv, solver_call_
figures/ # PDFs mirrored into ../paper/imgs/
```

Quickstart

```
“bash cd experiments pip install -r requirements.txt
```


**(optional) download SVRPBench; on failure
the pipeline falls back to synthetic**

```
python -m src.download_data --config config.yaml
```


tiny smoke run to validate the pipeline (1 instance, ~5 events)

```
python -m src.run_experiments --config config.yaml --smoke
```


full run

```
python -m src.run_experiments --config config.yaml --scenario all
```


build the figures referenced in paper/root.tex

```
python -m src.plot_results ""
```

Reproducibility notes

- All randomness flows from `experiment.seed` in `config.yaml`.
- The same event stream is fed to **all three** scenarios (static, continuous, AET-RAG) so differences can be attributed to synchronization policy only.
- `rag.mode: oracle` runs without any external API: ground-truth severity is perturbed with controlled Gaussian noise to isolate the controller from LLM-side variability. Switch to `llm` (and export `OPENAI_API_KEY`) to obtain the RAG extraction-quality row of Table III.
- Hyperparameters (`theta_0`, `theta_min`, `gamma`, `weights`) are exposed in `config.yaml`; the sensitivity sweep is performed by editing this file or passing alternative configs.

Outputs that feed the paper

Paper artifact (root.tex)	Produced by	—————	—————
—	Table I tab:main_results	outputs/metrics/aggregate.csv	Table II tab:ablation
re-runs	with edited aet.weights/theta_t	Table III tab:rag_quality	re-runs with rag.mode in {oracle,
llm}	Fig. fig:solver_calls	plot_results.fig_solver_calls	Fig. fig:threshold_trace
plot_results.fig_threshold_trace	Fig. fig:tw_violations	plot_results.fig_tw_violations	
	Fig. fig:tradeoff	plot_results.fig_tradeoff	Fig. fig:sensitivity
sweep + custom	plotting		